Semana 5 — Clase Práctica en Laboratorio

Algoritmos de Búsqueda: Implementación, Optimización y Comparación Experimental

Introducción

Los problemas de esta semana ponen en práctica los **algoritmos de búsqueda** y la **notación O grande** presentados en la clase teórica. El eje temático es la implementación correcta, la optimización y la **comparación experimental** de distintas estrategias de búsqueda, midiendo tiempos de ejecución con el módulo `time` de Python y generando datos de prueba con el módulo `random`.

Cada problema sigue requiriendo la aplicación rigurosa de la **metodología de Pólya** (Análisis → Diseño → Codificación → Evaluación) y el **modelo de programa modular** de la Semana 4: cada algoritmo de búsqueda se implementa como una **función** con sección declarativa (docstring), precondiciones y postcondiciones.

Los seis problemas están organizados en tres bloques con **conexiones deliberadas**: los Problemas 1 y 2 implementan y validan los algoritmos fundamentales (búsqueda lineal y búsqueda binaria); los Problemas 3 y 4 incorporan las optimizaciones de búsqueda lineal (Move-to-Front, Transposición) y la búsqueda por interpolación; el Problema 5 diseña un experimento comparativo completo con generación pseudoaleatoria de datos y medición de tiempos; el Problema 6 es un desafío de extracursado que analiza el impacto de la distribución de datos en la búsqueda por interpolación.

> ***Recordatorio: Modelo de programa modular*** *— Cada función de búsqueda incluye su sección declarativa como docstring (descripción, precondición, postcondición), accesible mediante* `help()`*. La sección algorítmica se organiza en Prólogo / Resolución / Epílogo.*

> ***Módulo `random` de Python*** *— Para los problemas de esta semana se utilizarán las funciones* `random.randint(a, b)` *(entero aleatorio en [a, b]),* `random.sample(population, k)` *(k elementos sin repetición),* `random.choices(population, k=n)` *(n elementos con repetición) y* `random.shuffle(lista)` *(reordenar in-place). Para resultados reproducibles, fijar la semilla con* `random.seed(valor)` *al inicio del programa.*

Bloque A: Implementación y validación de algoritmos de búsqueda

Problema 1: Búsqueda lineal y búsqueda binaria

Enunciado

Implementar dos funciones de búsqueda en Python: `busqueda_lineal(datos, clave)` y `busqueda_binaria(datos, clave)`. Ambas reciben una secuencia y una clave, y retornan el índice de la clave en la secuencia o −1 si no se encuentra. La búsqueda binaria debe documentar como precondición que la secuencia está ordenada.

A continuación, escribir un programa que:

(a) Genere una lista ordenada de *n* enteros distintos usando `random.sample(range(1, 10 * n), n)` seguido de `sort()`, con *n* = 1000. Fijar la semilla con `random.seed(42)`.

(b) Para cada función de búsqueda, busque 10 claves: 5 presentes en la lista (elegidas con `random.sample(datos, 5)`) y 5 ausentes (enteros negativos). Verificar que ambas funciones retornan los mismos resultados.

(c) Informar para cada búsqueda: la clave buscada, el índice retornado y la cantidad de comparaciones realizadas (agregar un contador dentro de cada función).

Casos de análisis y prueba

Caso	Clave	Resultado esperado	Observación
Normal	Elemento interior	Índice correcto	Ambas funciones coinciden
Normal	Entero negativo	−1	Ausente en la lista
Límite	Primer elemento	0	Mejor caso lineal
Límite	Último elemento	n−1	Peor caso lineal si existe
Extremo	Lista vacía	−1	n = 0

Orientaciones para la resolución

Análisis: El problema tiene dos componentes: la implementación correcta de ambas funciones y la verificación cruzada de resultados. La búsqueda binaria requiere que la lista esté ordenada; la lineal no.

Diseño: Cada función incluye un parámetro opcional `contar` (booleano) que, cuando es `True`, retorna una tupla `(indice, comparaciones)` en lugar de solo el índice. Para la búsqueda lineal, usar un bucle `while` con condición compuesta y cortocircuito (Semana 3). Para la binaria, usar el esquema iterativo con `izq`, `der` y `medio`.

Evaluación: Verificar con los casos de la tabla. Además, confirmar que para toda clave presente, `datos[resultado] == clave`. Para claves ausentes, confirmar que la búsqueda lineal realiza exactamente *n* comparaciones.

> ***Reflexión: correctitud antes que eficiencia*** *— La primera prioridad es que ambas funciones retornen resultados correctos para todos los casos de prueba. La optimización y la medición de tiempos vienen después. Un error clásico en la búsqueda binaria es usar* `izq < der` *en lugar de* `izq <= der`*, lo que omite el caso de región con un solo elemento.*

Problema 2: Búsqueda binaria recursiva

Enunciado

Implementar una versión recursiva de la búsqueda binaria: `busqueda_binaria_rec(datos, clave, izq, der)`. La función recibe además los límites de la región de búsqueda y se invoca recursivamente reduciendo la región a la mitad.

Escribir un programa que:

(a) Reutilice la lista ordenada del Problema 1 (misma semilla, mismo *n*).

(b) Busque las mismas 10 claves del Problema 1 con la versión recursiva y verifique que los resultados son idénticos a los de la versión iterativa.

(c) Para tres valores de *n* (100, 1000, 10000), medir la profundidad máxima de recursión alcanzada (usar un parámetro `nivel` que se incrementa en cada llamada) y compararla con $\lceil \log_2(n) \rceil$.

Orientaciones para la resolución

Diseño: Dos casos base: (1) `izq > der` → retornar −1; (2) `datos[medio] == clave` → retornar `medio`. Dos casos recursivos: buscar en la mitad izquierda o derecha. La invocación inicial es `busqueda_binaria_rec(datos, clave, 0, len(datos) - 1)`.

Evaluación: Para *n* = 1000, la profundidad máxima debe ser ≤ 10 (ya que $\lceil \log_2(1000) \rceil = 10$). Si la profundidad es mayor, hay un error en la reducción de la región.

> ***Conexión con la Semana 4*** *— Cada llamada recursiva apila un nuevo frame en la pila de ejecución con sus propias copias de* `izq`, `der` *y* `medio`*. La profundidad máxima O(log n) implica un uso de memoria proporcional a* $\log_2(n)$ *frames, lo que para n = 1.000.000 son solo ~20 frames: un uso muy modesto de la pila.*

Bloque B: Optimizaciones y búsqueda por interpolación

Problema 3: Heurísticas de búsqueda lineal adaptativa

Enunciado

Implementar tres variantes de búsqueda lineal como funciones separadas:

(a) `busqueda_lineal(datos, clave)` — versión básica, sin reorganización.

(b) `busqueda_lineal_mtf(datos, clave)` — con heurística *Move-to-Front*: al encontrar la clave, moverla a la posición 0.

(c) `busqueda_lineal_transpose(datos, clave)` — con heurística de *Transposición*: al encontrar la clave, intercambiarla con el elemento inmediatamente anterior.

Las tres funciones deben retornar el índice donde se encontró la clave (o −1) **y** la cantidad de comparaciones realizadas.

A continuación, escribir un programa experimental que:

(a) Cree una lista inicial de 200 elementos distintos: `datos = list(range(200))`.

(b) Genere una secuencia de **1000 búsquedas** donde un subconjunto pequeño de claves se busca con mucha más frecuencia que el resto, por ejemplo `claves_frecuentes = random.choices(datos, k=5)` para un conjunto de 5 claves frecuentes (experimentar

cambiando la cantidad de claves frecuentes). Usar `random.choices(claves_frecuentes, k=700)` para las 700 búsquedas frecuentes y `random.choices(range(200), k=300)` para las 300 restantes. Mezclar ambas listas con `random.shuffle`.

(c) Ejecutar las 1000 búsquedas con cada variante (sobre copias independientes de la lista original para las variantes con heurística) y acumular el **total de comparaciones** de cada una.

(d) Informar: total de comparaciones por variante y porcentaje de reducción respecto de la búsqueda lineal básica: `(1-comp_`*`variante`*`/comp_lineal)*100`.

Resultado esperado (orientativo)

Variante	Comparaciones (aprox.)	Reducción vs. básica
Lineal básica	~100.000	—
Move-to-Front	~30.000 – 50.000	50–70%
Transposición	~60.000 – 80.000	20–40%

Los valores exactos dependerán de la semilla aleatoria y de la cantidad de claves frecuentes. Lo importante es que MTF y Transposición muestren una reducción significativa cuando hay **localidad temporal** en las búsquedas.

Orientaciones para la resolución

Análisis: La clave del experimento es que la distribución de búsquedas no es uniforme: 5 claves concentran el 70% de las búsquedas. Esta asimetría es lo que las heurísticas aprovechan.

Diseño: Usar `datos.copy()` para crear copias independientes antes de cada serie de búsquedas (las heurísticas modifican la lista in-place). Para MTF, usar `pop(i)` seguido de `insert(0, elemento)`. Para Transposición, un swap: `datos[i], datos[i-1] = datos[i-1], datos[i]`.

> ***Reflexión: localidad temporal y reorganización*** *— Move-to-Front se adapta inmediatamente pero es volátil: una búsqueda aislada de una clave infrecuente desplaza a las frecuentes. Transposición es más estable: las claves frecuentes ascienden gradualmente y resisten perturbaciones puntuales. La elección depende de si la distribución de búsquedas es estable o cambiante.*

Problema 4: Búsqueda por interpolación vs. búsqueda binaria

Enunciado

Implementar la función `busqueda_interpolacion(datos, clave)` según el algoritmo presentado en la clase teórica. La función debe retornar el índice (o −1) y la cantidad de comparaciones realizadas.

Escribir un programa que compare búsqueda binaria vs. interpolación en **tres escenarios** con listas de n = 10.000 elementos:

Escenario A — Distribución uniforme: generar la lista con `datos = sorted(random.sample(range(1, 100001), n))`. Buscar 500 claves presentes elegidas al azar.

Escenario B — Distribución exponencial (sesgada): los valores están concentrados cerca de cero con cola larga. Generar la lista garantizando n elementos únicos:

```
vals = set()
while len(vals) < n:
    vals.add(int(random.expovariate(1) * 10000))
datos = sorted(vals)
```

Buscar 500 claves presentes elegidas al azar con `random.sample(datos, 500)`.

Escenario C — Distribución con clusters: 5 grupos de 2000 elementos separados por vacíos. Generar la lista con exactamente $n/5$ elementos únicos por cluster:

```
centros = [5000, 25000, 50000, 75000, 95000]
vals = set()
for c in centros:
    cluster = set()
    while len(cluster) < n // 5:
        cluster.add(random.randint(c - 2500, c + 2500))
    vals |= cluster
datos = sorted(vals)
```

Buscar 500 claves presentes elegidas al azar. Los grandes vacíos entre clusters (~15.000–20.000 unidades) provocan estimaciones erráticas en la búsqueda por interpolación.

Para cada escenario, informar: promedio de comparaciones por búsqueda para cada algoritmo, y cuál es más eficiente.

Resultado esperado (orientativo)

Escenario	Binaria (comp. promedio)	Interpolación (comp. promedio)	Ganador
A: Uniforme	~13	~3–4	Interpolación
B: Exponencial	~13	~10–50+	Binaria
C: Clusters	~13	~5–20	Depende del cluster

Orientaciones para la resolución

Análisis: La búsqueda por interpolación estima la posición asumiendo distribución uniforme de valores. Cuando los datos son efectivamente uniformes (Escenario A), la estimación es excelente y el algoritmo converge en muy pocas comparaciones. Cuando la distribución es sesgada (Escenario B), la estimación es mala y puede requerir muchos pasos. La búsqueda binaria, en cambio, siempre divide por la mitad sin importar la distribución.

Diseño: Usar `random.seed(42)` al inicio para reproducibilidad. La función `busqueda_interpolacion` debe manejar el caso especial `datos[izq] == datos[der]` para evitar la división por cero. Las 500 claves presentes se eligen con `random.sample(datos, 500)`.

Evaluación: Antes de medir comparaciones, verificar que ambas funciones retornan los mismos índices para las mismas claves. Si difieren, hay un error de implementación.

> ***Conexión con la jerarquía de memoria*** *— Cuando los datos residen en disco, cada comparación implica un acceso costoso (~5 ms). En ese escenario, reducir las comparaciones de 13 a 3 (interpolación en distribución uniforme) equivale a reducir el tiempo de acceso de 65 ms a 15 ms: una diferencia perceptible por el usuario.*

Bloque C: Comparación experimental completa

Problema 5: Laboratorio de medición de tiempos de búsqueda

Enunciado

Diseñar y ejecutar un **experimento completo** que mida los tiempos de ejecución de búsqueda lineal, búsqueda binaria y búsqueda por interpolación para distintos tamaños de entrada.

El programa debe:

(a) Para cada *n* en [5000, 10000, 50000, 100000], generar una lista ordenada de *n* enteros distintos con distribución uniforme: `datos = sorted(random.sample(range(1, 10 * n), n))`.

(b) Generar 1000 claves a buscar: 500 presentes (elegidas con `random.sample(datos, 500)`) y 500 ausentes (elegidas con `random.sample(set(range(1, 10 * n)) - set(datos), 500)`).

(c) Para cada algoritmo y cada *n*, medir el **tiempo total** de las 1000 búsquedas usando `time.perf_counter()` (antes y después del bloque de búsquedas, y luego informando la diferencia después–antes). Repetir la medición 3 veces y promediar para reducir la variabilidad.

(d) Presentar una tabla de resultados con el formato:

n	Lineal (ms)	Binaria (ms)	Interpolación (ms)	Ratio Lin/Bin
5000	...	...	...	...
10.000	...	...	...	...
50.000	...	...	...	...
100.000	...	...	...	...

(e) Analizar: ¿el ratio Lineal/Binaria crece cuando *n* se multiplica por 10? ¿Qué relación tiene esto con O(n)/O(log n)?

Orientaciones para la resolución

Análisis: El objetivo es verificar empíricamente lo que la notación O grande predice teóricamente. Si la búsqueda lineal es O(n) y la binaria es O(log n), al multiplicar *n* por 10 el tiempo lineal debería multiplicarse por ~10, mientras que el tiempo binario debería crecer en una cantidad aproximadamente constante ($\log_2(10) \approx 3.3$ comparaciones adicionales).

Diseño: Estructura modular recomendada:

```
def generar_datos(n, semilla):        # Lista ordenada + claves de prueba
```

```
def medir_busquedas(funcion, datos, claves, repeticiones=3):
    # Retorna tiempo promedio en ms
def mostrar_tabla(resultados):       # Formato alineado


# Programa principal
for n in [100, 500, 1000, 5000, 10000, 50000, 100000]:
    datos, claves = generar_datos(n, semilla=42)
    t_lin = medir_busquedas(busqueda_lineal, datos, claves)
    t_bin = medir_busquedas(busqueda_binaria, datos, claves)
    t_int = medir_busquedas(busqueda_interpolacion, datos, claves)
```

Evaluación: Verificar que los tiempos de la búsqueda lineal crecen aproximadamente de forma lineal con *n*, que la binaria crece de forma logarítmica y que la interpolación se comporta mejor que la binaria en distribución uniforme. Si la lineal para *n* = 100.000 no tarda significativamente más que para *n* = 1000, el cronómetro no tiene suficiente resolución: aumentar la cantidad de búsquedas o de repeticiones.

> ***Reflexión: teoría vs. práctica*** *— Los tiempos medidos incluyen factores que la notación O grande ignora: el overhead del intérprete Python, los efectos de la caché del procesador, la recolección de basura y la carga del sistema operativo. Por eso los tiempos no serán proporciones exactas de las funciones teóricas, pero las tendencias de crecimiento deben coincidir.*

Bloque D: Desafío de extracursado

Problema 6: Impacto de la distribución en la búsqueda por interpolación

Enunciado

Diseñar un experimento que demuestre cuantitativamente cómo la **distribución de los datos** afecta el rendimiento de la búsqueda por interpolación. Para n = 10.000, generar cuatro listas con distribuciones diferentes y medir comparaciones y tiempos:

Distribución 1 — Uniforme: `sorted(random.sample(range(1, 100001), n))`.

Distribución 2 — Cuadrática: `sorted(set(int(random.random()**2 * 100000) for _ in range(2*n)))[:n]`. Los valores están concentrados cerca de cero.

Distribución 3 — Logarítmica: `sorted(set(int(math.log(1 + random.random() * (math.e**10 - 1)) / 10 * 100000) for _ in range(2*n)))[:n]`. Los valores están concentrados cerca del máximo.

Distribución 4 — Bimodal: `sorted(set(random.gauss(25000, 5000) for _ in range(n//2)) | set(random.gauss(75000, 5000) for _ in range(n//2)))`. Luego convertir a enteros. Dos concentraciones con un vacío central.

Para cada distribución, buscar 500 claves presentes y registrar el **promedio y máximo de comparaciones** de la búsqueda por interpolación y de la búsqueda binaria. Presentar los resultados en una tabla.

Preguntas para el análisis

(a) ¿En cuáles distribuciones la interpolación supera a la binaria? ¿En cuáles es peor?

(b) ¿Cuál es la relación entre el **máximo** de comparaciones de la interpolación y el tamaño n? ¿En algún caso se acerca a n?

(c) Si se desconoce la distribución de los datos, ¿qué algoritmo de búsqueda es la elección más segura? Justificar con los datos del experimento.

(d) Proponer un criterio cuantitativo que, dada una lista ordenada, permita decidir automáticamente si conviene usar interpolación o binaria. *Sugerencia: calcular una medida de uniformidad de la distribución, como la varianza de las diferencias consecutivas datos[i+1] − datos[i].* La **varianza** es una medida estadística que cuantifica la dispersión o variabilidad de un conjunto de datos respecto a su media aritmética; en términos simples, indica qué tan alejados o próximos están los valores individuales del promedio del grupo.

> ***Conexión con la clase teórica*** *— Este problema ejercita la idea central de la Sección 6.4 del documento teórico: la eficiencia de la búsqueda por interpolación depende críticamente de la distribución. El caso O(log log n) solo se garantiza para distribuciones uniformes; en el peor caso, la interpolación degrada a O(n). La búsqueda binaria ofrece O(log n) independientemente de la distribución: es la opción robusta.*

Notas finales

Estos seis problemas cubren los contenidos prácticos de la **Semana 5** del curso. Los Problemas 1 y 2 validan la implementación correcta de los algoritmos fundamentales de búsqueda, el Problema 3 ejercita las heurísticas de reorganización para búsqueda lineal adaptativa, el

Problema 4 explora la búsqueda por interpolación y su sensibilidad a la distribución, y el Problema 5 diseña un experimento de medición de tiempos completo. El Problema 6 es un desafío de extracursado que profundiza en el análisis experimental.

Criterios de evaluación formativa

Además de los criterios de semanas anteriores (metodología de Pólya, modelo de programa, *literate programming*, contratos de funciones), esta semana se evaluarán especialmente:

Correctitud verificada: cada función de búsqueda retorna resultados correctos para todos los casos de prueba antes de medir tiempos.

Diseño experimental: los experimentos fijan la semilla aleatoria para reproducibilidad, usan tamaños de muestra suficientes y promedian múltiples repeticiones.

Análisis crítico: las conclusiones sobre eficiencia se apoyan en datos medidos, no en intuiciones, y se contrastan con las predicciones de la notación O grande.

Modulación: cada algoritmo está encapsulado en una función con contrato; el programa experimental es un módulo separado que importa las funciones de búsqueda.

Para profundizar

(1) En el Problema 1, agregar un contador de comparaciones a la búsqueda binaria y verificar que para $n = 1024$ (potencia de 2) el peor caso es exactamente 11 comparaciones ($\lceil \log_2(1024) \rceil + 1$). (2) En el Problema 3, experimentar con una distribución de búsquedas que cambia a mitad de la secuencia: las primeras 500 búsquedas favorecen las claves {10, 20, 30} y las siguientes 500 favorecen {150, 160, 170}. ¿Cuál heurística se adapta mejor al cambio? (3) En el Problema 5, incluir en la tabla la búsqueda lineal con Move-to-Front y comparar su tiempo con las tres búsquedas originales para la distribución de claves del Problema 3 (no uniforme). (4) Implementar una versión de búsqueda por interpolación que, cuando detecta que la estimación de posición está lejos de la posición real, recurra a búsqueda binaria en la subregión restante (*interpolation-binary hybrid*). Comparar con la interpolación pura en el Escenario B del Problema 4. (5) Investigar el módulo `timeit` de Python como alternativa a `time.perf_counter()` para mediciones más precisas en microbenchmarks.